

Questions

1. What is the main purpose of the Command pattern?
 - A. Reduce memory usage
 - B. Encapsulate a request as an object
 - C. Allow runtime inheritance
 - D. Modify system performance
2. In the Command pattern, the object that performs the actual operation is the:
 - A. Invoker
 - B. Originator
 - C. Receiver
 - D. Adaptee
3. Which class in the example acts as an Invoker?
 - A. Command
 - B. MenuItem
 - C. CapitalizeCommand
 - D. TextDocument
4. Which method must be implemented by all Command classes?
 - A. Load()
 - B. Execute()
 - C. Draw()
 - D. Save()
5. What feature is enabled by adding the UnExecute() method in commands?
 - A. Delayed loading
 - B. Undo functionality
 - C. Logging
 - D. Parallel processing
6. In the example, CapitalizeCommand manipulates:
 - A. Graphics
 - B. Memento
 - C. TextDocument
 - D. Facade
7. Which component stores the command history?
 - A. Adapter
 - B. Memento
 - C. CommandProcessor
 - D. Proxy
8. What is the role of the Memento pattern?
 - A. Simplify the UI
 - B. Capture and restore object state
 - C. Execute stored commands
 - D. Control access to resources
9. Who creates a Memento?
 - A. Proxy
 - B. Receiver
 - C. Originator
 - D. Facade

10. What is stored inside a Memento?
 - A. State
 - B. Command reference
 - C. Log data
 - D. GUI layout
11. Which class restores state from a Memento?
 - A. Facade
 - B. Originator
 - C. Proxy
 - D. Invoker
12. What is the role of the Caretaker in the Memento pattern?
 - A. Modifies document state
 - B. Invokes execute
 - C. Stores Mementos
 - D. Creates adapters
13. What design pattern converts one interface to another?
 - A. Proxy
 - B. Adapter
 - C. Command
 - D. Memento
14. In the Adapter example, which class is being adapted?
 - A. Panel
 - B. TextDocument
 - C. TextBox
 - D. Receiver
15. EditableTextShape is an example of:
 - A. Receiver
 - B. Proxy
 - C. Adaptee
 - D. Adapter
16. What method does the Adapter use to return the bounding box?
 - A. Execute()
 - B. GetBoundingBox()
 - C. Load()
 - D. SetExtent()
17. What is the main purpose of the Facade pattern?
 - A. Capture object state
 - B. Provide simplified access to complex subsystems
 - C. Ensure type safety
 - D. Duplicate object behavior
18. Which of the following is NOT a goal of the Facade pattern?
 - A. Simplify usage
 - B. Unify interfaces
 - C. Increase subsystem coupling
 - D. Hide complexity
19. Which of the following is treated uniformly in the Composite pattern?
 - A. Commands
 - B. GUI windows

- C. Leaf and Composite objects
 - D. Text formatting
20. In the Composite pattern, what type of structure is typically used?
- A. Matrix
 - B. Array
 - C. Tree
 - D. Grid
21. In Composite, the container node that holds other graphics is called:
- A. Leaf
 - B. Panel
 - C. Adapter
 - D. Proxy
22. What method is used to render graphics in the Composite pattern?
- A. Load()
 - B. Draw()
 - C. Save()
 - D. Undo()
23. The Proxy pattern is used to:
- A. Hide implementation of complex subsystems
 - B. Provide direct access to a RealSubject
 - C. Control access and perform additional logic
 - D. Avoid inheritance
24. In the Proxy example, which object is responsible for delaying image loading?
- A. Image
 - B. ImageProxy
 - C. Panel
 - D. Adapter
25. Which method in ImageProxy avoids loading the actual image?
- A. Load()
 - B. Execute()
 - C. GetSize()
 - D. GetState()
26. What does the Proxy pattern often add?
- A. GUI support
 - B. Undo support
 - C. Additional behavior before and after request
 - D. RealSubject complexity
27. In the Command pattern, which of the following acts as a client?
- A. TextEditorApp
 - B. Memento
 - C. ImageProxy
 - D. Adapter
28. In the Memento pattern, restoring a document to a previous state is done via:
- A. Receiver
 - B. Load()
 - C. RestoreFromMemento()
 - D. Draw()

29. Which design pattern allows new commands to be added without modifying existing classes?
- A. Composite
 - B. Command
 - C. Facade
 - D. Memento
30. Which pattern allows treating a group of objects and individual objects uniformly?
- A. Adapter
 - B. Proxy
 - C. Composite
 - D. Command

Answer Key

1. B
2. C
3. B
4. B
5. B
6. C
7. C
8. B
9. C
10. A
11. B
12. C
13. B
14. C
15. D
16. B
17. B
18. C
19. C
20. C
21. C
22. B
23. B
24. C
25. B
26. C
27. A
28. C
29. B
30. C

Multiple correct answers Questions

1. Which of the following are components of the Command pattern?
☐ Receiver ☐ Invoker ☐ Command ☐ Facade
2. In the Command pattern, which elements might directly call Execute()?
☐ Receiver ☐ CommandProcessor ☐ MenuItem ☐ TextEditorApp
3. Which are examples of ConcreteCommand classes?
☐ OpenCommand ☐ TextDocument ☐ PasteCommand ☐ MenuItem
4. The Memento pattern involves which of the following participants?
☐ Originator ☐ Invoker ☐ Memento ☐ Caretaker
5. What methods are typically found in a Memento-enabled Originator class?
☐ GetState() ☐ RestoreFromMemento() ☐ CreateMemento() ☐ Execute()
6. Which patterns are used to support undo functionality?
☐ Command ☐ Proxy ☐ Memento ☐ Adapter
7. In the Adapter pattern, which classes might be involved?
☐ Adapter ☐ Adaptee ☐ Client ☐ Composite
8. Which are true about the EditableTextShape class?
☐ Implements Shape ☐ Uses delegation ☐ Wraps a TextBox ☐ Acts as Receiver
9. The Facade pattern aims to:
☐ Provide a unified interface ☐ Increase subsystem complexity ☐ Hide underlying implementation ☐ Simplify client interaction
10. Composite pattern components typically include:
☐ Leaf ☐ Command ☐ Composite ☐ Component (Interface)
11. Which classes can be part of a Composite structure in the lecture example?
☐ Line ☐ Rect ☐ Panel ☐ CommandProcessor
12. What methods are defined by the Shape interface in the Adapter pattern?
☐ CreateManipulator() ☐ GetBoundingBox() ☐ Draw() ☐ Paste()
13. What are uses of the Proxy pattern?
☐ Logging ☐ Lazy initialization ☐ Access control ☐ Undo support
14. In the Proxy example, which methods are overridden by the Proxy?
☐ Draw() ☐ Execute() ☐ GetSize() ☐ Load()
15. The CommandProcessor is responsible for:
☐ Storing commands ☐ Executing commands ☐ Restoring command state ☐ Creating GUIs
16. Which are types of proxies?
☐ Remote ☐ Virtual ☐ Facade ☐ Protective
17. In the Command pattern, the Receiver performs:
☐ The actual operation ☐ GUI rendering ☐ State saving ☐ Text manipulation
18. Which commands support undo/redo in the examples?
☐ CapitalizeCommand ☐ ClearCommand ☐ OpenCommand ☐ PasteCommand
19. Which components of the Command pattern know about the Receiver?
☐ ConcreteCommand ☐ CommandProcessor ☐ MenuItem ☐ Facade

20. What does the Caretaker store in the Memento pattern?
☐ Memento ☐ Command history ☐ Originator reference ☐ State
21. Facade pattern benefits include:
☐ Reducing complexity ☐ Making subsystems easier to use ☐ Providing detailed access
☐ Hiding implementation details
22. What are benefits of the Composite pattern?
☐ Tree structures ☐ Uniform operations ☐ Strong coupling ☐ Recursive behavior
23. Which are correct roles in the Adapter pattern?
☐ Target ☐ Adaptee ☐ Adapter ☐ Originator
24. In the Memento pattern, what is true about the Originator?
☐ Creates Mementos ☐ Restores state ☐ Executes GUI actions ☐ Wraps commands
25. What are goals of the Proxy pattern?
☐ Reduce object creation ☐ Enforce access control ☐ Enhance performance ☐ Act as Receiver
26. In Composite, which can be a Composite or Leaf?
☐ Component ☐ Operation ☐ Panel ☐ Text
27. What is typically stored in a Memento?
☐ Document text ☐ Cursor position ☐ GUI state ☐ File name
28. What can be delegated in Adapter?
☐ Method calls ☐ State saving ☐ Requests ☐ Object instantiation
29. Which pattern enables commands to be queued and executed later?
☐ Command ☐ Memento ☐ Proxy ☐ Adapter
30. What are shared across all design patterns discussed?
☐ Object roles ☐ Abstraction ☐ Decoupling ☐ Strong inheritance

Answer Key

1. Receiver, Invoker, Command
2. CommandProcessor, MenuItem, TextEditorApp
3. OpenCommand, PasteCommand
4. Originator, Memento, Caretaker
5. RestoreFromMemento(), CreateMemento()
6. Command, Memento
7. Adapter, Adaptee, Client
8. Implements Shape, Uses delegation, Wraps a TextBox
9. Provide a unified interface, Hide underlying implementation, Simplify client interaction
10. Leaf, Composite, Component (Interface)
11. Line, Rect, Panel
12. CreateManipulator(), GetBoundingBox()
13. Logging, Lazy initialization, Access control
14. Draw(), GetSize(), Load()
15. Storing commands, Executing commands
16. Remote, Virtual, Protective
17. The actual operation, Text manipulation
18. CapitalizeCommand, ClearCommand
19. ConcreteCommand
20. Memento
21. Reducing complexity, Making subsystems easier to use, Hiding implementation details
22. Tree structures, Uniform operations, Recursive behavior
23. Target, Adaptee, Adapter
24. Creates Mementos, Restores state
25. Reduce object creation, Enforce access control, Enhance performance
26. Component, Panel, Text
27. Document text, Cursor position
28. Method calls, Requests
29. Command
30. Object roles, Abstraction, Decoupling

True or false Questions

1. The Command pattern decouples the object that invokes an operation from the one that knows how to perform it. **(T/F)**
2. In the Command pattern, the Execute() method is defined only in the Receiver class. **(T/F)**
3. The MenuItem in the TextEditorApp acts as the Invoker in the Command pattern. **(T/F)**
4. OpenCommand and PasteCommand are examples of concrete commands. **(T/F)**
5. The CommandProcessor class in the pattern stores commands for redo functionality only. **(T/F)**
6. The UnExecute() method is used to undo a previously executed command. **(T/F)**
7. The Memento pattern allows direct access to the internal state of an object. **(T/F)**
8. A TextDocMemento object is used to store the state of a TextDocument. **(T/F)**
9. In the Memento pattern, the Caretaker modifies the state stored in the Memento. **(T/F)**
10. The Originator is responsible for creating and restoring Mementos. **(T/F)**
11. In the Adapter pattern, the Adapter class implements the Adaptee interface. **(T/F)**
12. The EditableTextShape class adapts a TextBox to act like a Shape. **(T/F)**
13. The Adapter pattern is only useful when the source and target interfaces are similar. **(T/F)**
14. The Facade pattern exposes multiple interfaces from subsystems to the client. **(T/F)**
15. A Facade simplifies client interaction with complex subsystems. **(T/F)**
16. In the Composite pattern, both leaf and composite objects implement the same interface. **(T/F)**
17. A Panel in the Composite pattern can contain Line, Rect, and Text elements. **(T/F)**
18. Composite design allows you to treat individual and composite objects differently. **(T/F)**
19. The Draw() method is implemented only by composite nodes. **(T/F)**
20. In Proxy, a Proxy object controls access to a RealSubject. **(T/F)**
21. The Proxy pattern is never used to enhance performance. **(T/F)**
22. An ImageProxy can provide the image size without loading the actual image. **(T/F)**
23. Lazy initialization is a concept used in Proxy to delay expensive object creation. **(T/F)**
24. The Proxy pattern and the Adapter pattern serve the same purpose. **(T/F)**
25. A Proxy can perform additional behavior before or after forwarding a request. **(T/F)**
26. Command pattern is ideal for implementing operations like undo/redo. **(T/F)**
27. The Adapter pattern helps integrate new functionality without changing existing code. **(T/F)**
28. In Memento, the state is restored by calling CreateMemento() on the Memento object. **(T/F)**
29. The Command pattern can be extended to support macros (group of commands). **(T/F)**
30. Composite design pattern is useful for representing tree-like structures. **(T/F)**

Answer Key

1. True
2. False
3. True
4. True
5. False
6. True
7. False
8. True
9. False
10. True
11. False
12. True
13. False
14. False
15. True
16. True
17. True
18. False
19. False
20. True
21. False
22. True
23. True
24. False
25. True
26. True
27. True
28. False
29. True
30. True

Questions (One-Word Answer)

1. The Command pattern encapsulates a request as a(n) _____.
2. The interface in the Command pattern defines the method _____().
3. The object that executes the command is known as the _____.
4. A command that restores a previous state implements the method _____().
5. In the TextEditor example, the document being edited is the _____.
6. The object that invokes the command upon a user action is the _____.
7. The command that changes text to uppercase is called _____Command.
8. Commands are stored and managed by a _____Processor.
9. To support undo operations, we store command _____.
10. A command that clears document content is called _____Command.
11. The Memento pattern is used to capture and restore an object's _____.
12. In the Memento pattern, the object storing state is called a(n) _____.
13. The Memento is created by the _____.
14. In the Memento pattern, the object restoring its state is the _____.
15. The pattern that enables interface compatibility is called _____.
16. An adapter wraps an incompatible interface with a _____ interface.
17. The Adapter that uses composition is known as an _____ adapter.
18. The TextBox class is adapted to a Shape using a class called _____TextShape.
19. The method GetExtent() is used by the adapter to implement _____().
20. The pattern that provides a simplified interface to a subsystem is _____.
21. In the Facade pattern, the object hiding complexity is the _____.
22. The Composite pattern lets clients treat individual and composite objects _____.
23. In the Composite pattern, the container object is the _____.
24. In Composite, both leaves and composites implement the same _____.
25. In Proxy, the object controlling access is the _____.
26. A proxy may serve a request itself or delegate to the _____.
27. A proxy can delay the creation of an object, this is called _____ initialization.
28. The graphic class that simulates image loading in the Proxy example is _____Proxy.
29. A Panel in the Composite pattern can contain multiple _____.
30. In Command pattern, each command must implement the _____ method.

Answer Key

1. object
2. Execute
3. Receiver
4. UnExecute
5. TextDocument
6. Invoker
7. Capitalize
8. Command
9. history
10. Clear
11. state
12. Memento
13. Originator
14. Originator
15. Adapter
16. target
17. object
18. Editable
19. GetBoundingBox
20. Facade
21. Facade
22. uniformly
23. Composite
24. interface
25. Proxy
26. RealSubject
27. lazy
28. Image
29. Graphics
30. Execute